

# Graph Utils Documentation

This document describes all public utilities in `graph_utils.typ`.

## Table of Contents

|                                                                                              |    |
|----------------------------------------------------------------------------------------------|----|
| Graph Utils Documentation .....                                                              | 1  |
| 0.1. Compact Cheat Sheet .....                                                               | 1  |
| 0.2. Quick Start .....                                                                       | 2  |
| 0.3. String Utilities .....                                                                  | 2  |
| 0.3.1. <code>clip-str(txt, n)</code> .....                                                   | 2  |
| 0.3.2. <code>truncate-title(txt, enabled: true, max-chars: 18)</code> .....                  | 2  |
| 0.3.3. <code>fit-lines(txt, max-chars: 16, max-lines: 2)</code> .....                        | 3  |
| 0.3.4. <code>chars-cap(width, min: 8, scale: 4.2)</code> .....                               | 3  |
| 0.4. Geometry Utilities .....                                                                | 3  |
| 0.4.1. <code>node-size(node, outer: false)</code> .....                                      | 3  |
| 0.4.2. <code>node-edge(node, side: "right", outer: false, outer-value: none)</code> .....    | 3  |
| 0.4.3. <code>node-anchor(node, side: "right", outer: false, outer-value: none)</code> .....  | 3  |
| 0.4.4. <code>auto-pos-right(after, width, gap: 1.0, y: none, outer-after: true)</code> ..... | 3  |
| 0.4.5. <code>side-dir(side)</code> .....                                                     | 3  |
| 0.5. Node Constructors .....                                                                 | 4  |
| 0.5.1. <code>make-dataset(...)</code> .....                                                  | 4  |
| 0.5.2. <code>make-trapezoid(...)</code> .....                                                | 4  |
| 0.5.3. <code>make-box(...)</code> .....                                                      | 5  |
| 0.6. Drawing Helpers .....                                                                   | 6  |
| 0.6.1. <code>draw-node(node)</code> .....                                                    | 6  |
| 0.6.2. <code>make-arrow(...)</code> .....                                                    | 6  |
| 0.6.3. <code>draw-arrow(arr)</code> .....                                                    | 6  |
| 0.6.4. <code>edge-label(a, b, txt, ...)</code> .....                                         | 7  |
| 0.7. Security Emoji Utility .....                                                            | 7  |
| 0.7.1. <code>draw-node-emoji(...)</code> .....                                               | 7  |
| 0.8. Default Configurators .....                                                             | 8  |
| 0.8.1. <code>set-arrow-defaults(options: ())</code> .....                                    | 8  |
| 0.8.2. <code>set-dataset-defaults(options: ())</code> .....                                  | 8  |
| 0.8.3. <code>set-trapezoid-defaults(options: ())</code> .....                                | 8  |
| 0.8.4. <code>set-box-defaults(options: ())</code> .....                                      | 8  |
| 0.9. Complete Integrated Example .....                                                       | 9  |
| 0.10. Publishing Notes .....                                                                 | 10 |

## 0.1. Compact Cheat Sheet

- `make-dataset(...)`: stacked image dataset node.
- `make-trapezoid(...)`: encoder/decoder trapezoid node.
- `make-box(...)`: rectangular processing node.
- `draw-node(node)`: render any node.
- `make-arrow(from, to, ...)`: build routed arrow with labels.
- `draw-arrow(arr)`: render arrow route.
- `draw-node-emoji(node, ...)`: place lock/key/custom emoji near node.
- `set-arrow-defaults(...)`, `set-dataset-defaults(...)`, `set-trapezoid-defaults(...)`, `set-box-defaults(...)`: preconfigured constructors.

Most used one-liners:

```
#let ds = make-dataset("Image\nDataset", pos: (1, 0))
#let enc = make-trapezoid("FeatureEncoder", subtitle: "Encoder", after: ds, gap: 1.2)
#let arr = make-arrow(ds, enc, from-outer: true, auto-spacing: true, label: [input])
#draw-node(ds)
#draw-node(enc)
#draw-arrow(arr)
```

## 0.2. Quick Start

```
#import "@preview/cetz:0.4.2": canvas, draw
#import "graph_utils.typ": *

#let make-arrow = set-arrow-defaults(options: (spacing: 0.45, label-gap: 0.30))
#let make-trapezoid = set-trapezoid-defaults(options: (width: 2.8, big-half: 1.65,
small-half: 0.80))

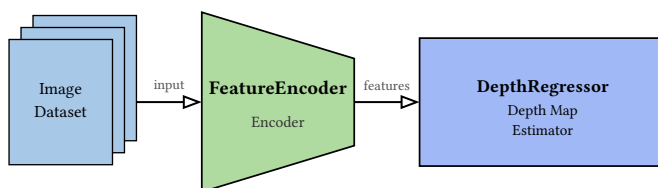
#canvas(length: 0.72cm, {
  import draw: *

  let ds = make-dataset("Image\nDataset", pos: (1.0, 0.0), images: 3)
  let enc = make-trapezoid("FeatureEncoder", subtitle: "Encoder", after: ds, gap:
1.2)
  let box = make-box("DepthRegressor", subtitle: "Depth Map\nEstimator", after: enc,
gap: 1.2)

  draw-node(ds)
  draw-node(enc)
  draw-node(box)

  let a = make-arrow(ds, enc, from-outer: true, auto-spacing: true, label: [input])
  let b = make-arrow(enc, box, auto-spacing: true, label: [features])
  draw-arrow(a)
  draw-arrow(b)
})
```

Rendered output:



## 0.3. String Utilities

### 0.3.1. clip-str(txt, n)

Clips a string to the first n grapheme clusters.

```
#clip-str("Computed Depth Maps", 8) // "Computed"
#clip-str("😄😄😄😄", 2) // keeps emoji boundaries
```

Rendered output: Computed, 😄😄

### 0.3.2. truncate-title(txt, enabled: true, max-chars: 18)

Truncates with ... when enabled.

```
#truncate-title("Very long title", enabled: true, max-chars: 8)
#truncate-title("Very long title", enabled: false)
```

Rendered output: Very ...,    Very long title

### 0.3.3. `fit-lines(txt, max-chars: 16, max-lines: 2)`

Current behavior: manual only (no auto wrap). If text contains `\n`, line breaks are preserved.

```
#fit-lines("Line 1\nLine 2")
#fit-lines("No auto wrapping applied")
```

Rendered output: Line 1

Line 2 / No auto wrapping applied

### 0.3.4. `chars-cap(width, min: 8, scale: 4.2)`

Computes estimated character capacity from width.

```
#chars-cap(2.0)
#chars-cap(4.0, min: 10, scale: 5.0)
```

Rendered output: 8,    20

## 0.4. Geometry Utilities

### 0.4.1. `node-size(node, outer: false)`

Returns (width, height) of a node; stack nodes include offset when `outer: true`.

```
#let ds = make-dataset("A", images: 3, image-size: (1.9, 2.3), image-spacing: 0.22)
#node-size(ds)
#node-size(ds, outer: true)
```

Rendered output: (1.9, 2.3),    (2.34, 2.7399999999999998)

### 0.4.2. `node-edge(node, side: "right", outer: false, outer-value: none)`

Returns one edge coordinate (x for left/right, y for top/bottom).

```
#let ds = make-dataset("A", images: 3, image-size: (1.9, 2.3), image-spacing: 0.22)
#node-edge(ds, side: "right")
#node-edge(ds, side: "right", outer: true)
#node-edge(ds, side: "right", outer: true, outer-value: 0.8)
```

Rendered output: 1.95,    2.39,    2.75

### 0.4.3. `node-anchor(node, side: "right", outer: false, outer-value: none)`

Returns (x, y) anchor for a node side.

```
#let box = make-box("B", size: (2.4, 1.8), pos: (5, 0))
#node-anchor(box, side: "left")
#node-anchor(box, side: "top")
```

Rendered output: (3.8, 0),    (5, 0.9)

### 0.4.4. `auto-pos-right(after, width, gap: 1.0, y: none, outer-after: true)`

Computes a position to place a node to the right of another one.

```
#let left = make-box("Left", size: (2.2, 1.6), pos: (2, 0))
#auto-pos-right(left, 3.0, gap: 1.25)
#auto-pos-right(left, 3.0, gap: 1.25, y: -1.0)
```

Rendered output: (5.85, 0),    (5.85, -1.0)

### 0.4.5. `side-dir(side)`

Direction unit vector for side names.

```
#side-dir("left")
#side-dir("right")
#side-dir("top")
#side-dir("bottom")
```

Rendered output:  $(-1.0, 0.0)$ ,  $(1.0, 0.0)$ ,  $(0.0, 1.0)$ ,  $(0.0, -1.0)$

## 0.5. Node Constructors

### 0.5.1. make-dataset(...)

Creates a stacked-image node.

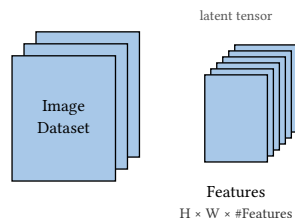
Important options:

- title, subtitle, legend, legend-position
- images, image-size, image-spacing
- title-position: inside or below
- title-truncate, max-title-chars
- position: pos or after + gap + optional y

```
#let ds1 = make-dataset(
  "Image\nDataset",
  images: 3,
  image-size: (1.9, 2.3),
  image-spacing: 0.22,
  pos: (1.0, 0.0),
)

#let ds2 = make-dataset(
  "Features",
  subtitle: "H × W × #Features",
  legend: "latent tensor",
  legend-position: "top",
  images: 6,
  image-size: (1.15, 1.6),
  image-spacing: 0.11,
  title-position: "below",
  after: ds1,
  gap: 1.2,
)
```

Rendered output:



### 0.5.2. make-trapezoid(...)

Creates encoder/decoder trapezoids.

Important options:

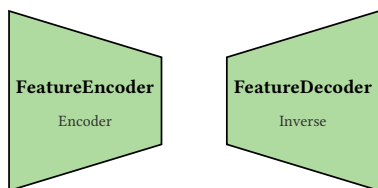
- mode: encoder or decoder
- width, big-half, small-half
- labels: title, subtitle, legend

- position: pos or after + gap

```
#let enc = make-trapezoid(
  "FeatureEncoder",
  subtitle: "Encoder",
  mode: "encoder",
  width: 2.8,
  big-half: 1.65,
  small-half: 0.80,
  pos: (5.0, 0.0),
)
```

```
#let dec = make-trapezoid(
  "Decoder",
  subtitle: "Inverse",
  mode: "decoder",
  width: 2.8,
  big-half: 1.65,
  small-half: 0.80,
  after: enc,
  gap: 1.2,
)
```

Rendered output:



### 0.5.3. make-box(...)

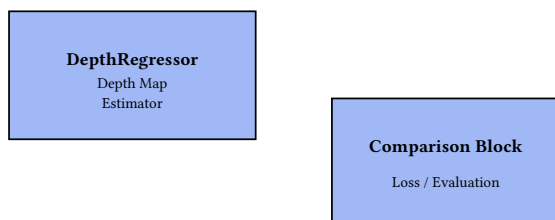
Creates rectangular blocks with optional auto size.

Important options:

- size: manual (w, h) or none for auto
- auto size controls: pad-x, min-w, min-h
- labels: title, subtitle, legend
- position: pos or after + gap + optional y

```
#let b1 = make-box("DepthRegressor", subtitle: "Depth Map\nEstimator", size: none,
pos: (10.0, 0.0))
#let b2 = make-box("Comparison Block", subtitle: "Loss / Evaluation", after: b1, gap:
1.4, y: -1.6)
```

Rendered output:



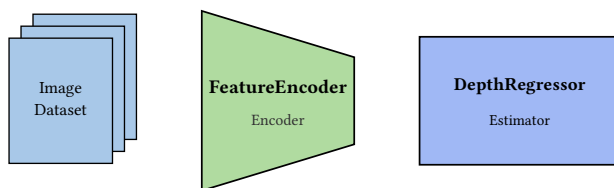
## 0.6. Drawing Helpers

### 0.6.1. draw-node(node)

Draws a node created by `make-dataset`, `make-trapezoid`, or `make-box`.

```
#canvas(length: 0.72cm, {
  import draw: *
  let ds = make-dataset("Image\nDataset", pos: (1.0, 0.0))
  let enc = make-trapezoid("FeatureEncoder", subtitle: "Encoder", after: ds, gap:
1.2)
  let box = make-box("DepthRegressor", subtitle: "Estimator", after: enc, gap: 1.2)
  draw-node(ds)
  draw-node(enc)
  draw-node(box)
})
```

Rendered output:



### 0.6.2. make-arrow(...)

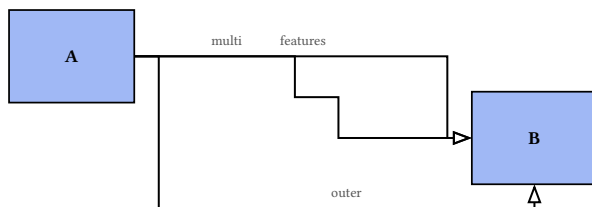
Creates orthogonal routes between nodes. Supports multi-segment routing.

Important options:

- side control: out-side, in-side
- outer anchors: from-outer, to-outer, from-outer-value, to-outer-value
- spacing: auto-spacing, spacing
- route mode: mode with h and v pattern (e.g., hv, vh, hvhvh)
- segment controls: mode-shift (legacy), mode-shifts (per-segment)
- labels: label, label-side, label-gap, label-dx, label-dy

```
#let a = make-arrow(node-a, node-b, mode: "hv", label: [features])
#let b = make-arrow(node-a, node-b, mode: "hvhvh", mode-shifts: (2.5, none, 0.8,
none, 1.2))
#let c = make-arrow(node-a, node-b, from-outer: true, in-side: "bottom", mode: "vh")
```

Rendered output:



### 0.6.3. draw-arrow(arr)

Draws an arrow route created by `make-arrow`.

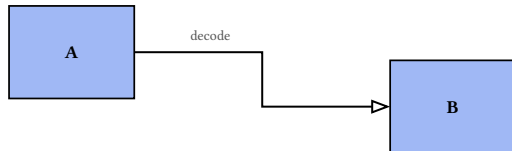
```
#canvas(length: 0.72cm, {
  import draw: *
  let left = make-box("A", pos: (2, 0))
  let right = make-box("B", pos: (9, -1))
  draw-node(left)
```

```

draw-node(right)
let arr = make-arrow(left, right, mode: "hvh", label: [decode])
draw-arrow(arr)
})

```

Rendered output:



#### 0.6.4. edge-label(a, b, txt, ...)

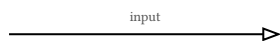
Draws a label along an explicit segment.

```

#canvas(length: 0.72cm, {
  import draw: *
  line((1,0), (6,0), mark: (end: ">"), stroke: (paint: black, thickness: 0.75pt))
  edge-label((1,0), (6,0), [input], side: "above", gap: 0.30)
})

```

Rendered output:



## 0.7. Security Emoji Utility

### 0.7.1. draw-node-emoji(...)

Draws lock/key/custom emoji around a node with optional status text.

Options:

- kind: lock-closed, lock-open, key, custom
- emoji: custom emoji content (overrides kind)
- place: top, bottom, left, right, over, under
- gap, shift, size, color
- use-outer: use full stacked footprint for placement
- key-state: encrypted or decrypted
- state-text, state-size, state-gap, state-position

```

#canvas(length: 0.72cm, {
  import draw: *
  let enc = make-trapezoid("FeatureEncoder", subtitle: "Encoder", pos: (5, 0))
  draw-node(enc)

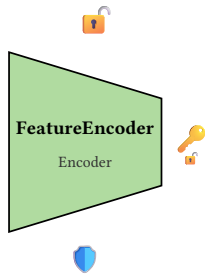
  // open lock above
  draw-node-emoji(enc, kind: "lock-open", place: "top", shift: (0.2, 0.0))

  // key on right with state
  draw-node-emoji(enc, kind: "key", place: "right", key-state: "decrypted")

  // custom emoji below
  draw-node-emoji(enc, kind: "custom", emoji: [🔒], place: "under")
})

```

Rendered output:



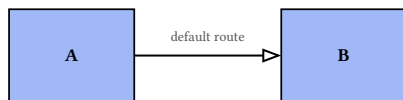
## 0.8. Default Configurators

### 0.8.1. `set-arrow-defaults(options: ())`

Returns a preconfigured make-arrow function.

```
#let make-arrow = set-arrow-defaults(options: (spacing: 0.5, label-gap: 0.35))
```

Rendered output:



### 0.8.2. `set-dataset-defaults(options: ())`

Returns a preconfigured make-dataset function.

```
#let make-dataset = set-dataset-defaults(options: (images: 4, image-spacing: 0.18))
```

Rendered output:

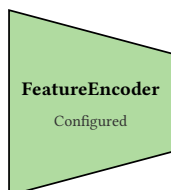


### 0.8.3. `set-trapezoid-defaults(options: ())`

Returns a preconfigured make-trapezoid function.

```
#let make-trapezoid = set-trapezoid-defaults(options: (width: 3.0, big-half: 1.7, small-half: 0.9))
```

Rendered output:



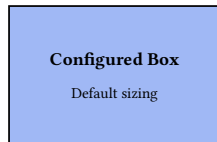
### 0.8.4. `set-box-defaults(options: ())`

Returns a preconfigured make-box function.

```
#let make-box = set-box-defaults(options: (min-w: 2.8, min-h: 1.9, pad-x: 0.75))
```

Rendered output:





## 0.9. Complete Integrated Example

```
#import "@preview/cetz:0.4.2": canvas, draw
#import "graph_utils.typ": *

#let node-gap = 1.2
#let make-arrow = set-arrow-defaults(options: (spacing: 0.45, label-gap: 0.30))
#let make-trapezoid = set-trapezoid-defaults(options: (width: 2.8, big-half: 1.65,
small-half: 0.80))

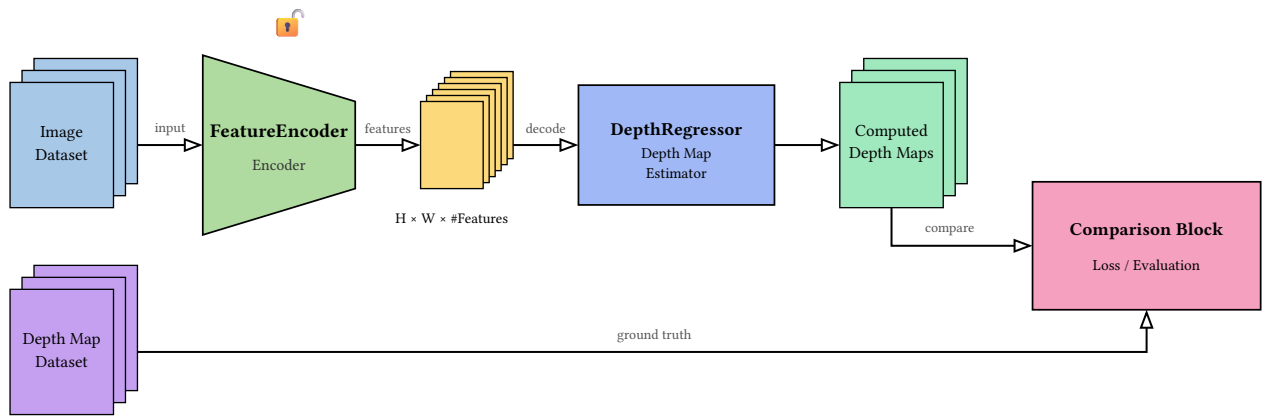
#canvas(length: 0.72cm, {
  import draw: *

  let image-ds = make-dataset("Image\nDataset", images: 3, pos: (1.0, 0.0), color:
rgb("#a8c8e8"))
  let depth-ds = make-dataset("Depth Map\nDataset", images: 3, pos: (1.0, -3.8),
color: rgb("#c5a0f0"))
  let encoder = make-trapezoid("FeatureEncoder", subtitle: "Encoder", after: image-
ds, gap: node-gap, color: rgb("#b0dba0"))
  let latent = make-dataset("H x W x #Features", images: 6, image-size: (1.15, 1.6),
image-spacing: 0.11, title-position: "below", after: encoder, gap: node-gap, color:
rgb("#fcd97a"), title-size: 0.50em)
  let detect = make-box("DepthRegressor", subtitle: "Depth Map\nEstimator", after:
latent, gap: node-gap, size: (3.6, 2.2), color: rgb("#a0b8f5"))
  let compds = make-dataset("Computed\nDepth Maps", images: 3, after: detect, gap:
node-gap, color: rgb("#a0e8be"))
  let cmp = make-box("Comparison Block", subtitle: "Loss / Evaluation", after:
compds, gap: node-gap, y: -1.85, color: rgb("#f5a0be"))

  draw-node(image-ds)
  draw-node(depth-ds)
  draw-node(encoder)
  draw-node(latent)
  draw-node(detect)
  draw-node(compds)
  draw-node(cmp)
  draw-node-emoji(encoder, kind: "lock-open", place: "top", shift: (0.2, 0.0))

  draw-arrow(make-arrow(image-ds, encoder, from-outer: true, auto-spacing: true,
label: [input]))
  draw-arrow(make-arrow(encoder, latent, auto-spacing: false, label: [features]))
  draw-arrow(make-arrow(latent, detect, from-outer: true, auto-spacing: false, label:
[decode]))
  draw-arrow(make-arrow(detect, compds, auto-spacing: false))
  draw-arrow(make-arrow(compds, cmp, out-side: "bottom", in-side: "left", auto-
spacing: true, mode: "vh", label: [compare]))
  draw-arrow(make-arrow(depth-ds, cmp, out-side: "right", in-side: "bottom", from-
outer: true, auto-spacing: true, mode: "hv", label: [ground truth]))
})
```

Rendered output:



## 0.10. Publishing Notes

1. Keep this file next to `graph_utils.typ`.
2. For package publication, include both files and add this document as your main README/guide.
3. The examples are designed to be copied directly into Typst documents.